Assignment 2 – Question 1: File System Management

1. Introduction

File system management is an important responsibility of an operating system. It allows users and applications to create, access, modify and delete files and directories.

This assignment demonstrates file system operations using Python on Parrot OS. The program performs basic file and directory management operations and implements a simple indexing technique using a Python dictionary to improve file lookup.

2. Objectives

The objectives of this implementation are to:

- Create and delete files and directories.
- Write and read data from files.
- Update existing file contents.
- Implement an indexing technique for file lookup.
- Demonstrate how an application interacts with the operating system's file system.

3. Programming Language and Tools

Item	Description
Programming language	Python 3
Operating system	Parrot OS (Linux)
Editor	Kate
Main module	`os`
Indexing technique	Dictionary-based indexing

4. Implementation

4.1 File and Directory Management

The program uses Python's built-in `os` module to interact with the file system.

The following operations are implemented:

Operation	Description
Create directory	Creates a new directory using `os.makedirs()`.
List directories	Displays directories using `os.walk()`.
Delete directory	Removes an empty directory using `os.rmdir()`.
Create file	Creates a new file using Python's `open()` function.

Write file	Writes content using write mode.
Read file	Retrieves and displays stored content.
Update file	Replaces existing text with new text.
Delete file	Removes a file using `os.remove()`.

The program stores its test files inside a directory named `file_system_demo` to keep the operations separate from other files.

4.2 Indexing Technique

The implementation uses a Python dictionary called `file_index` to store file paths.

For example:

```
file_index = {
    "Documents/notes.txt":
    "file_system_demo/Documents/notes.txt"
}
```

The dictionary stores the relative filename as the key and its full path as the value.

When a file operation is requested, the program looks up the filename in the dictionary instead of searching through all directories and files.

Dictionary lookups have an average time complexity of O(1), while a linear search through a collection of files has a time complexity of O(n).

The index is built when the program starts by scanning the demonstration directory using `os.walk()`.

This provides a simple demonstration of how indexing can improve file-access efficiency.

4.3 Program Execution

The program presents a menu that allows the user to select an operation.

The execution process is as follows:

1. The program creates the demonstration directory if it does not already exist.
2. It scans the directory and builds the file index.
3. It displays the available file management operations.
4. The user selects an operation.
5. The program performs the requested operation.
6. The menu continues to appear until the user chooses to exit.

4.4 Running the Program

The program is executed using: `python3 file_system.py`

5. Testing and Results

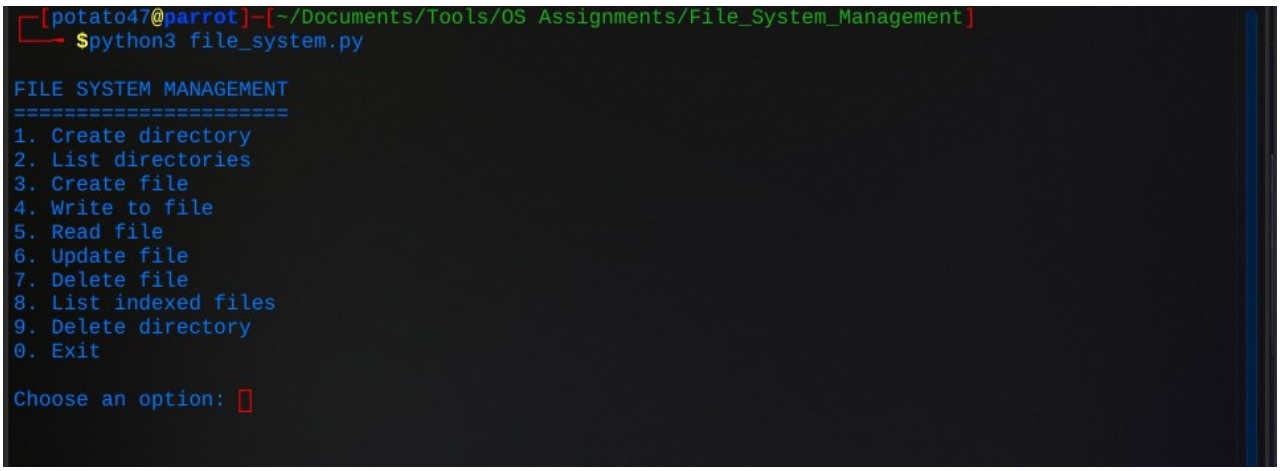
The implementation was tested by performing the different file and directory operations through the program menu.

5.1 Test Procedure

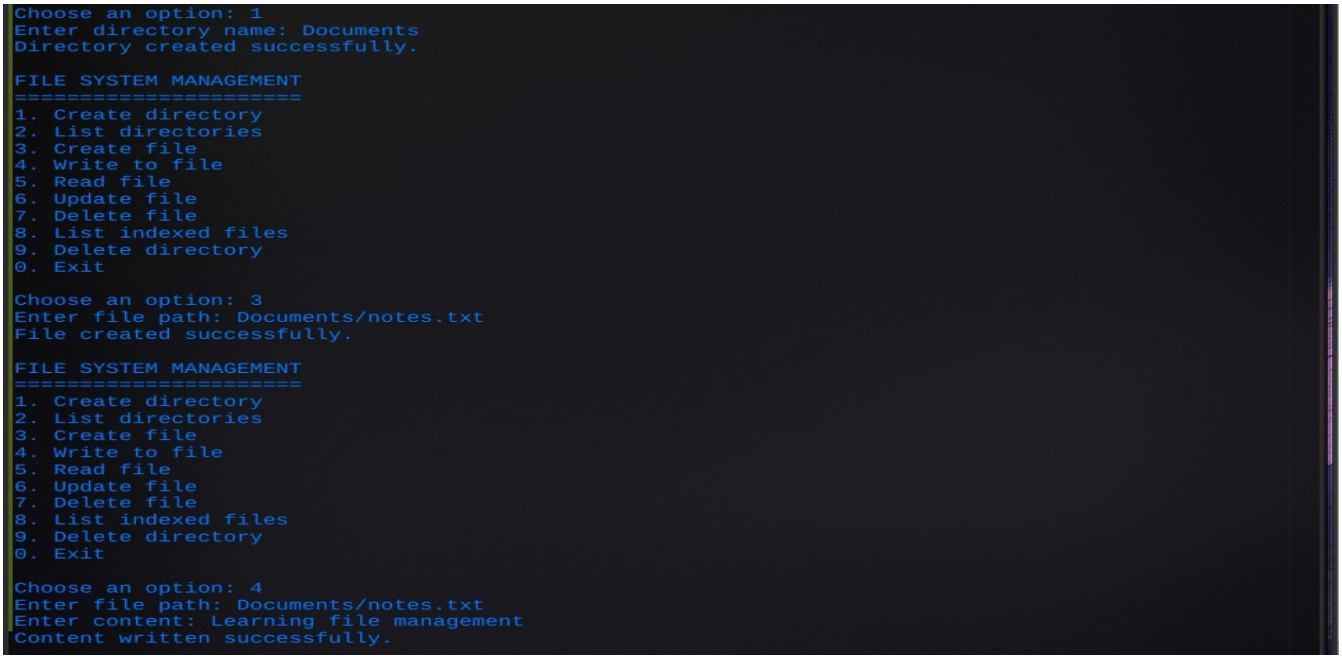
Test	Operation	Expected result
1	Create directory	A directory named `Documents` is created.
2	Create file	`Documents/notes.txt` is created.
3	Write file	Text is stored in the file.
4	Read file	The stored content is displayed.
5	Update file	Existing text is replaced with new text.
6	Read updated file	The modified content is displayed.
7	List indexed files	The file index displays the stored filename.
8	Delete file	The selected file is removed.
9	Delete directory	The empty `Documents` directory is removed.

5.2 Test Data

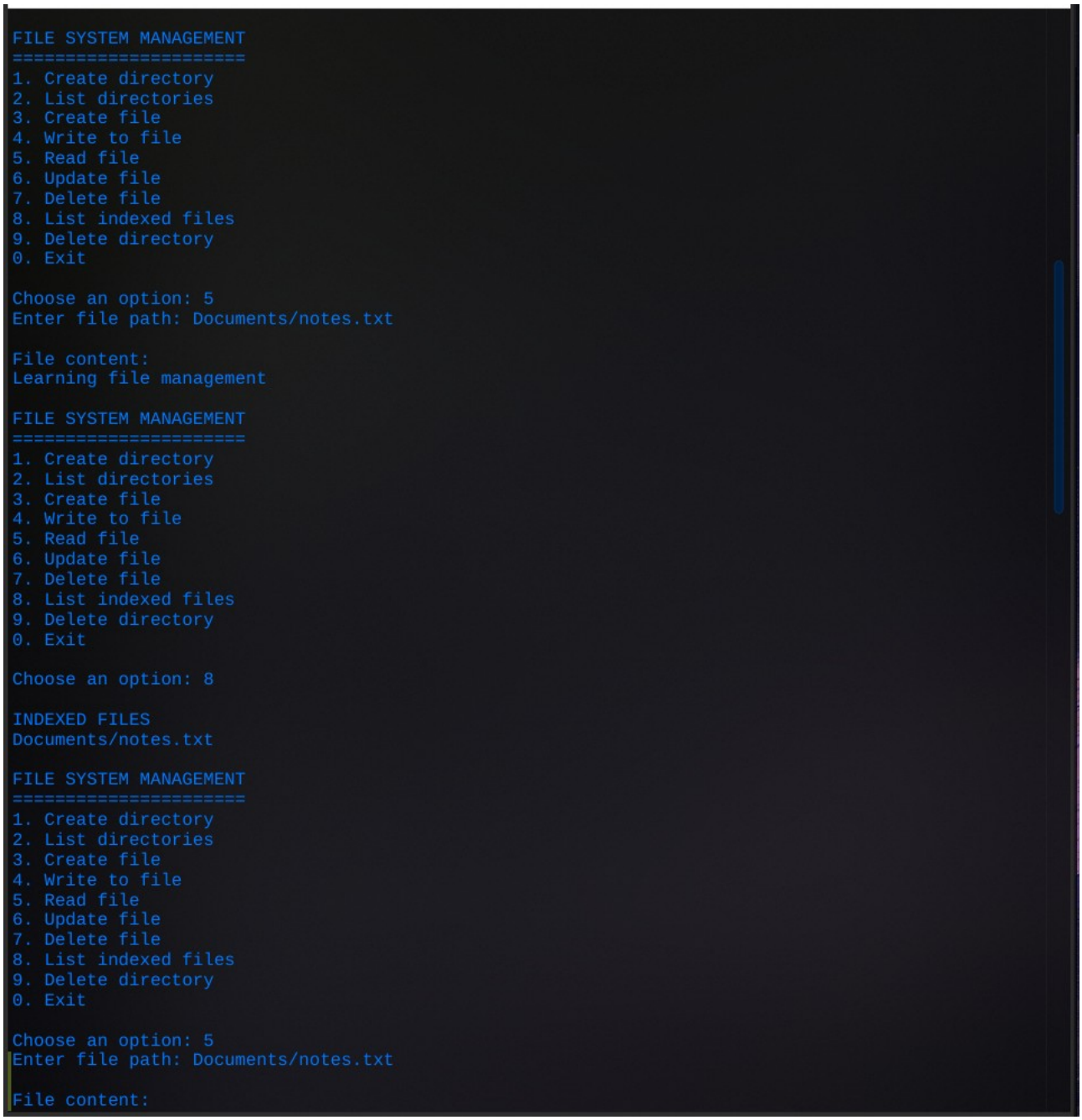
The following content was used during testing:



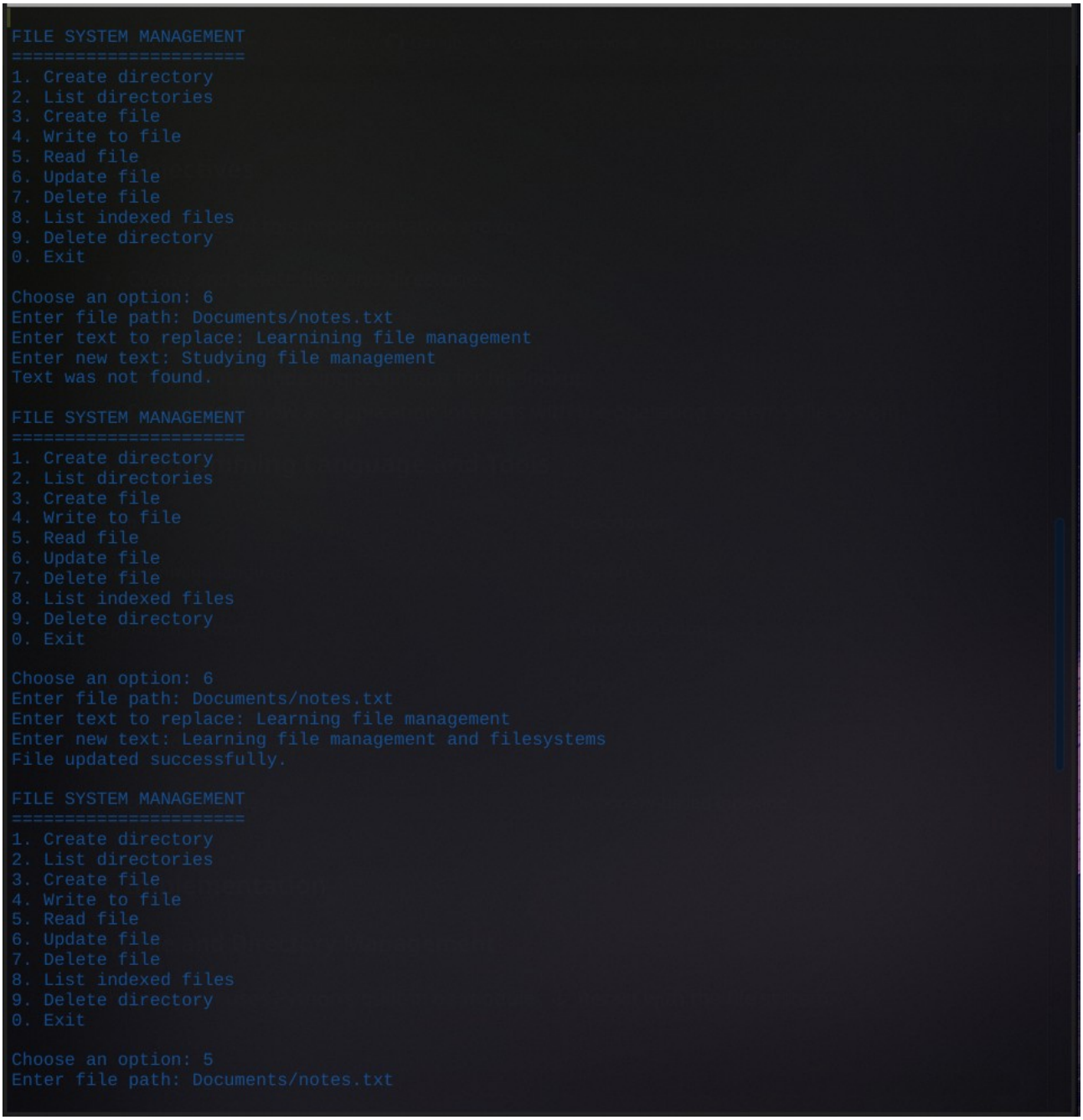
Creating a directory and a file



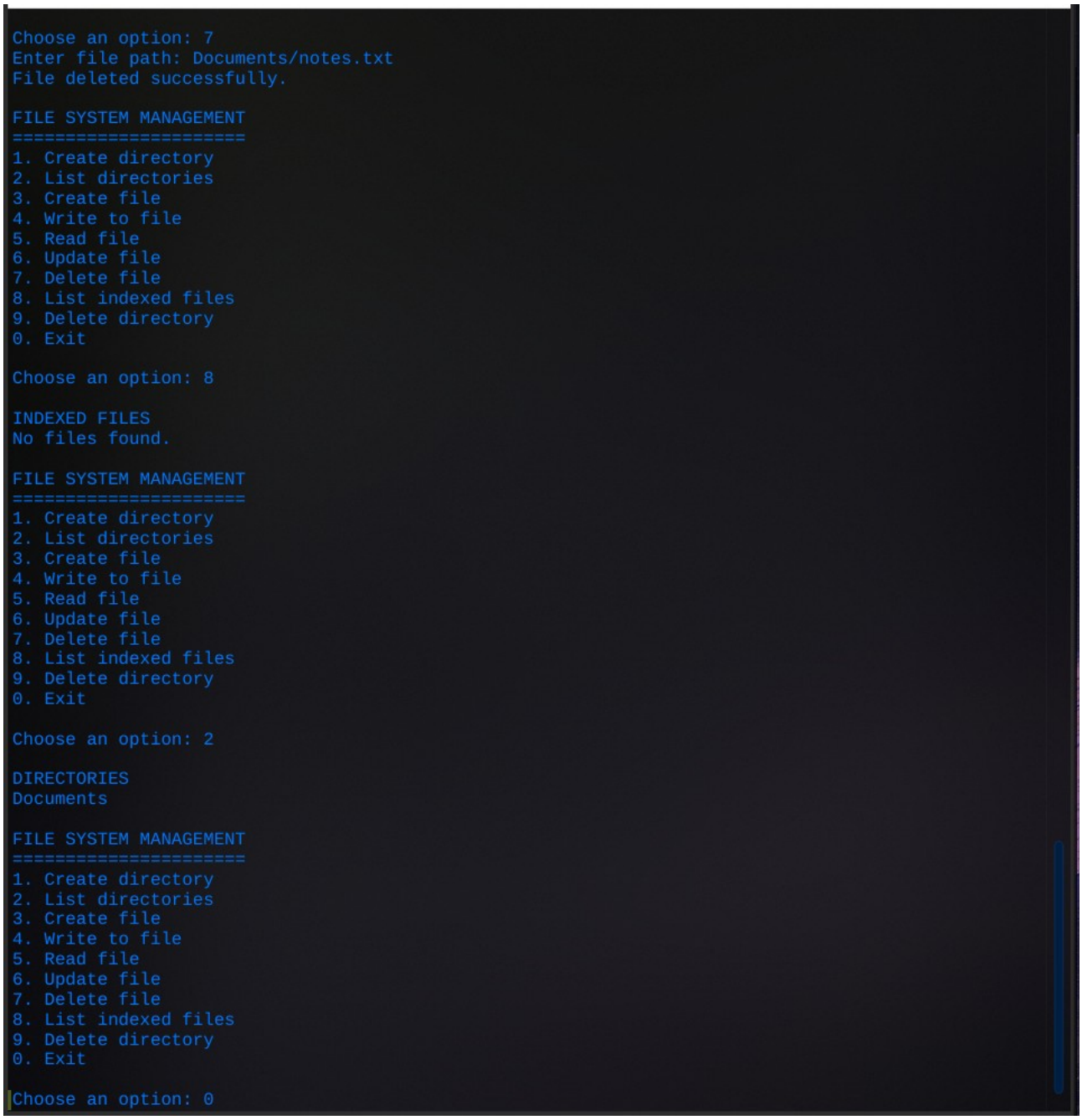
Reading, and Listing the indexed files



Updating and verifying if file is updated with new text



Listing indexe file sand deleting file



6. Discussion

The implementation demonstrates basic file system management operations using Python.

The use of the `os` module allows the application to interact with the operating system's file system without requiring direct hardware access.

The dictionary-based index improves file lookup by providing direct access to stored file paths. This reduces the need to repeatedly search through directory structures when locating indexed files.

However, the index is stored in memory and is rebuilt whenever the program starts. Changes made to files outside the program may therefore not be reflected in the index until it is rebuilt.

Another limitation is that the directory deletion function only removes empty directories. This prevents the program from accidentally deleting directories containing other files.

7. Conclusion

The assignment demonstrates how an application can interact with the operating system to manage files and directories.

The program implements file creation, reading, writing, updating and deletion, as well as directory management.

It also demonstrates a simple dictionary-based indexing technique that improves average file lookup efficiency.

The implementation provides a practical understanding of file system operations and the role of the operating system in managing stored data.

8. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*. Wiley.
2. Python Software Foundation. `os — Miscellaneous operating system interfaces`. https://docs.python.org/3/library/os.html
3. Python Software Foundation. `Built-in Functions — open()`. https://docs.python.org/3/library/functions.html#open
4. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*. https://pages.cs.wisc.edu/~remzi/OSTEP/